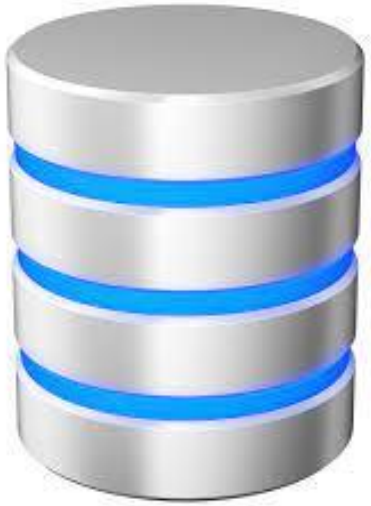


Chapter2:
relational data model

Relational data model

Relational data model was developed for structuring the data for sound database processing

The relational data model consists of the following two components

Data structure

Data are organized in the form of tables, with rows and columns.

Data manipulation

Powerful operations (typically implemented using the SQL language) are used to manipulate data stored in the relations

Relation

❑Definition: A relation is a named, two-dimensional table of data

❑ a table of records (also called tuples) and consists of

a relation schema and relation instance

❑ schema for a relation specifies its

name and the name, the type, and the domain of each field in the table (or column, or attribute)

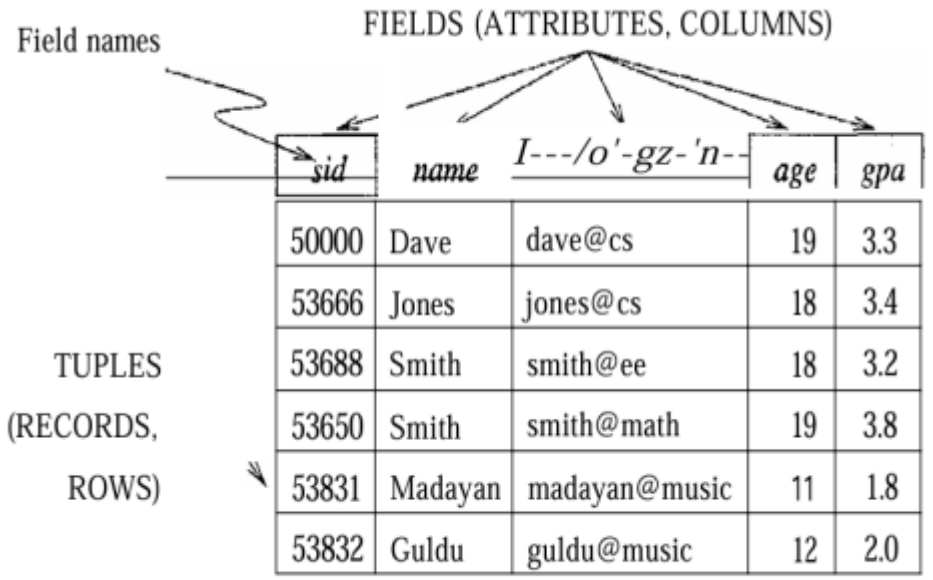


Fig. 2.1 An instance of student Relation

the sechma of the student relation

Students (*sid*: string, *name*: string, *login*: string, *age*: integer, *gpa*: real)

PROPERTIES OF RELATIONS

- Requirements:
  - Every relation has a unique name.
  - Every attribute value is atomic (not multivalued, not composite)
  - Every row is unique (can't have two rows with exactly the same values f or all their fields)
  - Attributes (columns) in tables have unique names
  - The order of the columns is irrelevant
  - The order of the rows is irrelevant
  - The Degree of a relation is the number of fields it contains.
  - The Cardinality of a relation instance is the number of tuples it contains.

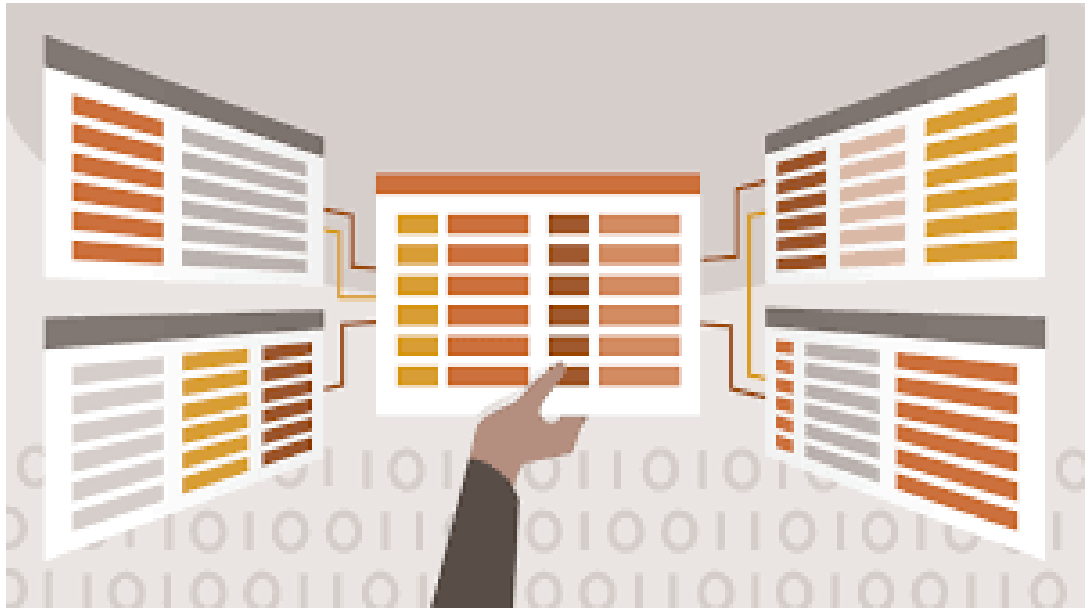
Key Fields

- Keys are special fields that serve two main purposes:
  - ***Primary keys*** are unique identifiers of the relation.
  - Examples include employee numbers, social security numbers, etc. *This is how we can guarantee that all rows are unique*
  - ***Foreign keys*** are identifiers that enable a dependent relation (on the many side of a relationship) to refer to its parent relation (on the one side of the relationship)
- Keys can be ***simple*** (a single field) or ***composite*** (more than one field)
- Keys usually are used as indexes to speed up the response to user queries.

A relational database

A relational database may consist of any number of relations.

The structure of the database is described through using of a schema which is a description of the overall logical structure of the database.



Relational Database Schema

- The term "schema" refers to the organization of data as a blueprint of how the database is constructed (divided into database tables in the case of relational databases).

- Database schema is Just a representation and it does not contain any data.

- Database schema is not expected to change frequently

- The actual data in a database may change quite frequently.

Database schema(cont.)

CUSTOMER

CustomerID	CustomerName	CustomerAddress	CustomerCity*	CustomerState*	CustomerPostalCode

ORDER

OrderID	OrderDate	CustomerID

ORDER LINE

OrderID	ProductID	OrderedQuantity

PRODUCT

ProductID	ProductDescription	ProductFinish	ProductStandardPrice	ProductLineID

Customer_T

Customer_ID	Customer_Name	Customer_Address	Customer_City	Customer_State	Postal_Code
1	Contemporary Casuals	1355 S Hines Blvd	Gainesville	FL	326012871
2	Value Furniture	15145 S.W. 17th St.	Plano	TX	750947743
3	Home Furnishings	1900 Allard Ave.	Albany	NY	122091125
4	Eastern Furniture	1925 Beltline Rd.	Carteret	NJ	070083188
5	Impressions	5585 Westcott Ct.	Sacramento	CA	942064056

Order_T

Order_ID	Order_Date	Customer_ID
1001	10/21/2008 12:00:00 AM	1
1010	11/5/2008 12:00:00 AM	1
1006	10/24/2008 12:00:00 AM	2
1005	10/24/2008 12:00:00 AM	3
1009	11/5/2008 12:00:00 AM	4

Order_Line_T

Order_ID	Product_ID	Ordered_Quantity
1001	1	2
1001	2	2
1001	4	1
1002	3	5
1003	3	3

expressing a schema

- CUSTOMER(CustomerID: string, CustomerName: string, CustomerAddress: string, CustomerCity: string, CustomerState: string, CustomerPostalCode: string)
- ORDER(OrderID: string, OrderDate:date, CustomerID: string)
- ORDER LINE(OrderID: string, ProductID: string, OrderedQuantity:real)
- PRODUCT(ProductID: string, ProductDescription: string, ProductFinish: string, ProductStandardPrice:real, ProductLineID: string)

CUSTOMER

CustomerID	CustomerName	CustomerAddress	CustomerCity*	CustomerState*	CustomerPostalCode

ORDER

OrderID	OrderDate	CustomerID

ORDER LINE

OrderID	ProductID	OrderedQuantity

PRODUCT

ProductID	ProductDescription	ProductFinish	ProductStandardPrice	ProductLineID

A graphical representation

expressing a schema: sample university database,

Students (*sid*: string, *name*: string, *login*: string, *age*: integer, *gpa*: real)
Faculty (*fid*: string, *fname*: string, *sal*: real)
Department (*dept_name*: string, *building*: string, *budget*: real)
Courses (*cid*: string, *cname*: string, *credits*: integer)
Rooms (*rno*: integer, *address*: string, *capacity*: integer)
Enrolled (*studid*: string, *cid*: string, *grade*: string)
Teaches (*fid*: string, *cid*: string)
Meets_In (*cid*: string, *rno*: integer, *fname*: string)

Short text statements

conceptual schema VS physical schema

In a Relational DBMS (RDBMS),

- the conceptual schema is the Relational Database Schema
- The physical schema of the relational model summarizes how the relations described in the conceptual schema are actually stored on secondary storage devices such as disks and tapes
- Decisions about the physical schema are based on an understanding of how the data is typically accessed
- The process of arriving at a good physical schema is called physical database design.

conceptual schema VS physical schema

A sample physical schema

- what **file organizations** to use to store the relations and create **a uxiliary data structures**, called **indexes** to speed up data retriev al operations
- Decisions about the physical schema are based on an understan ding of how the data is typically accessed
  - For example :Store all relations as unsorted files of records. (A file in a DBMS is either a collection of records or a collection of pages
  - Create indexes on the first column of required relation

External schema

- Any given database has one exactly one conceptual schema and one physical schema
- it may have several external schemas
- Each external schema consists of a collection of one or more views and relations from the conceptual schema
- A view is conceptually a relation defined by a query
- it does not store data physically like a table.
- Instead, it presents data from one or more tables dynamically
- A user can treat a view just like a relation and ask questions about the records in the view
    - A **materialized view** is a **pre-computed and stored copy** of data from one or more tables (or views). the database **runs the query once** and **saves the result** (like a table).

Examples

1. A university database contains information about Students, Courses, a nd Enrollments. Define the relational schema for the following informatio n:

Each student has a unique Student_ID, Name, and Major. Each course has a unique Course_ID, Course_Name, and Credits. A student can enroll in many courses. For each enrollment, store the Grade.

Answer1: STUDENT(Student_ID, Name, Major)

→ Primary Key: Student_ID

COURSE(Course_ID, Course_Name, Credits)

→ Primary Key: Course_ID

ENROLLMENT(Student_ID, Course_ID, Grade)

→ Primary Key: (Student_ID, Course_ID)

Examples…

- **BOOK**(Book_ID, Title, Publisher, Year)
→ Primary Key: Book_ID
- **AUTHOR**(Author_ID, Name)
→ Primary Key: Author_ID
- **BOOK_AUTHOR**(Book_ID, Author_ID)
→ Primary Key: (Book_ID, Author_ID)
→ Foreign Keys:
- Book_ID → BOOK(Book_ID)
- Author_ID → AUTHOR(Author_ID)
- **BORROWER**(Borrower_ID, Name, Address)
→ Primary Key: Borrower_ID
- **BORROWING**(Borrower_ID, Book_ID, Borrow_Date, Return_Date)
→ Primary Key: (Borrower_ID, Book_ID, Borrow_Date)
→ Foreign Keys:
- Borrower_ID → BORROWER(Borrower_ID)
- Book_ID → BOOK(Book_ID)

2. Create a relational schema for a library system that stores information about Books, Authors, and Borrowers. Each book can have one or more authors, and a borrower can borrow many books.

Examples…

3. A university keeps data about **Professors**, **Students**, **Research Projects**, and **Departments**:

- Each department has Dept_ID, Dept_Name, and Building.
- Each professor has Prof_ID, Name, Rank.
- Each student has Student_ID, Name, Degree.
- A research project has Proj_ID, Title, Budget, and is led by one professor.
- students can **work** on many projects, and for each, record the Role (e.g., "Leader", "Assistant", "Researcher").

Design a relational schema with keys and relationships

Examples :answer3

DEPARTMENT(Dept_ID, Dept_Name, Building)

PROFESSOR(Prof_ID, Name, Rank, Dept_ID)

STUDENT(Student_ID, Name, Degree, Dept_ID)

PROJECT(Proj_ID, Title, Budget, Lead_Prof)

WORKS_ON(Proj_ID, Person_ID, Role)

example

A company database has data about **employees**, **departments**, **projects and dependent**:

•Each department has a unique Dept_ID and a Dept_Name. each department has a manger, can mange many projects

•Each employee has a unique Emp_ID, a Name,salary ,adderess and belongs to one department.

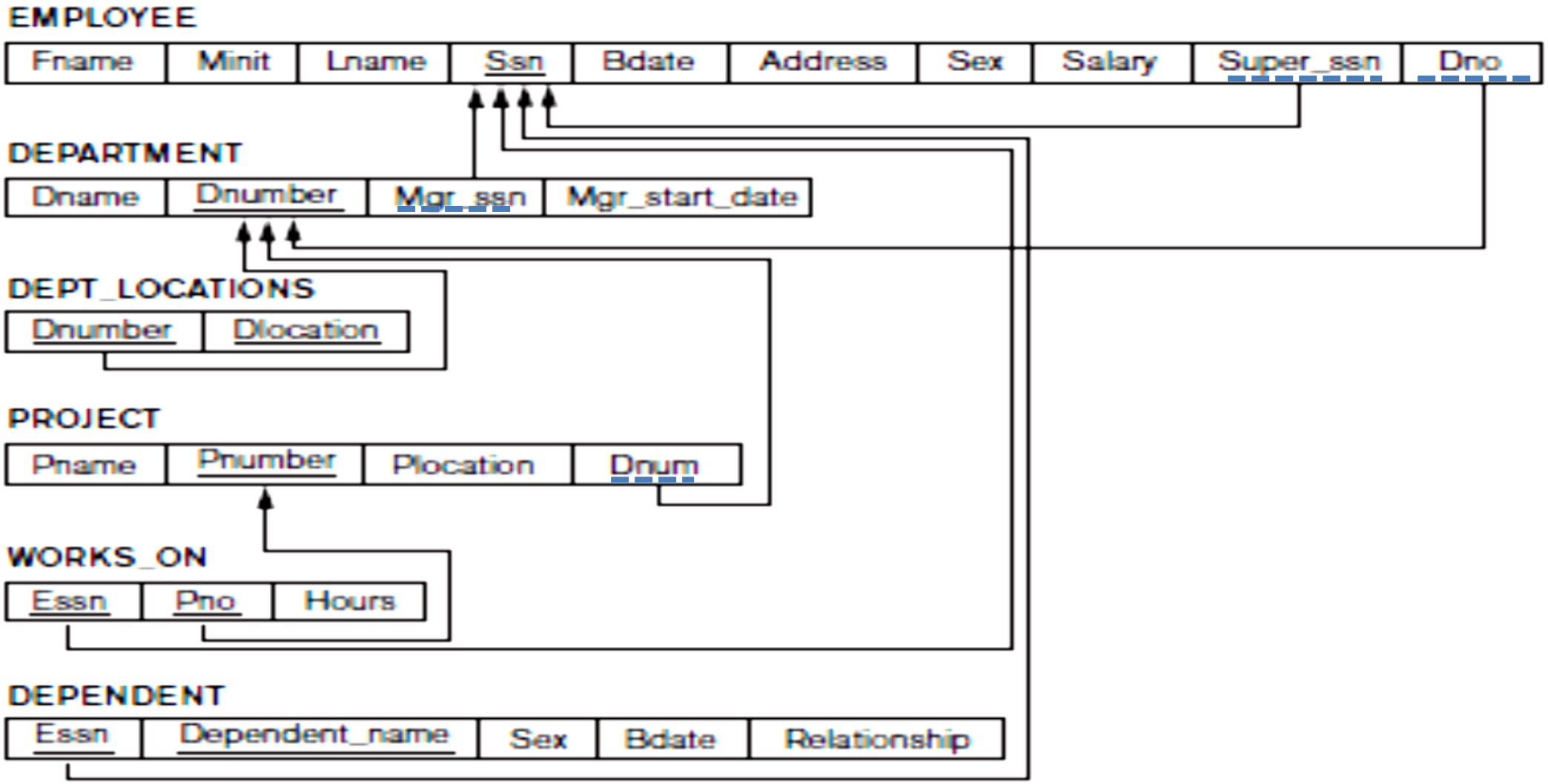
•Each project has a unique Proj_ID, a Proj_Name, and may involve multiple employees. Under control on only department

•Each employees has many dependent. Each dependent as name , sex, degree of relation ship and birthdate

•Each employee has a supervisor

Design the relational schema and specify **keys** and **relationships**.

Example: relational model for company



Integrity Constraints(IC)

- IC are conditions specified on a database schema and restricts the data that can be stored in an instance of the database.

- The relational data model includes several types of constraints, or rules limiting acceptable values and actions, whose purpose is to facilitate maintaining the accuracy and integrity of data in the database

- legal instance: If a database instance satisfies all the integrity constraints specified on the database schema

Kinds of Integrity Constraints

- Domain Constraints
  - A domain is the set of values that may be assigned to an attri bute.
  - The value that appear in a column must be drawn from the d omain associated with that column

  - Formally, an instance of a relation R with the schema

R (f1: D1, ..., fn: Dn) that satisfies the domain constraint is a set of tuples (records) with n fields, such that {< f1: d1, ..., fn: dn> | d1 ϵ Dom1, ..., dn ϵ Domn}.

Kinds of Integrity Constraints

❑Primary Key Constraint

(candidate) Key : a certain minimal subset of fields of a relation is a unique ident ifier for a tuple.

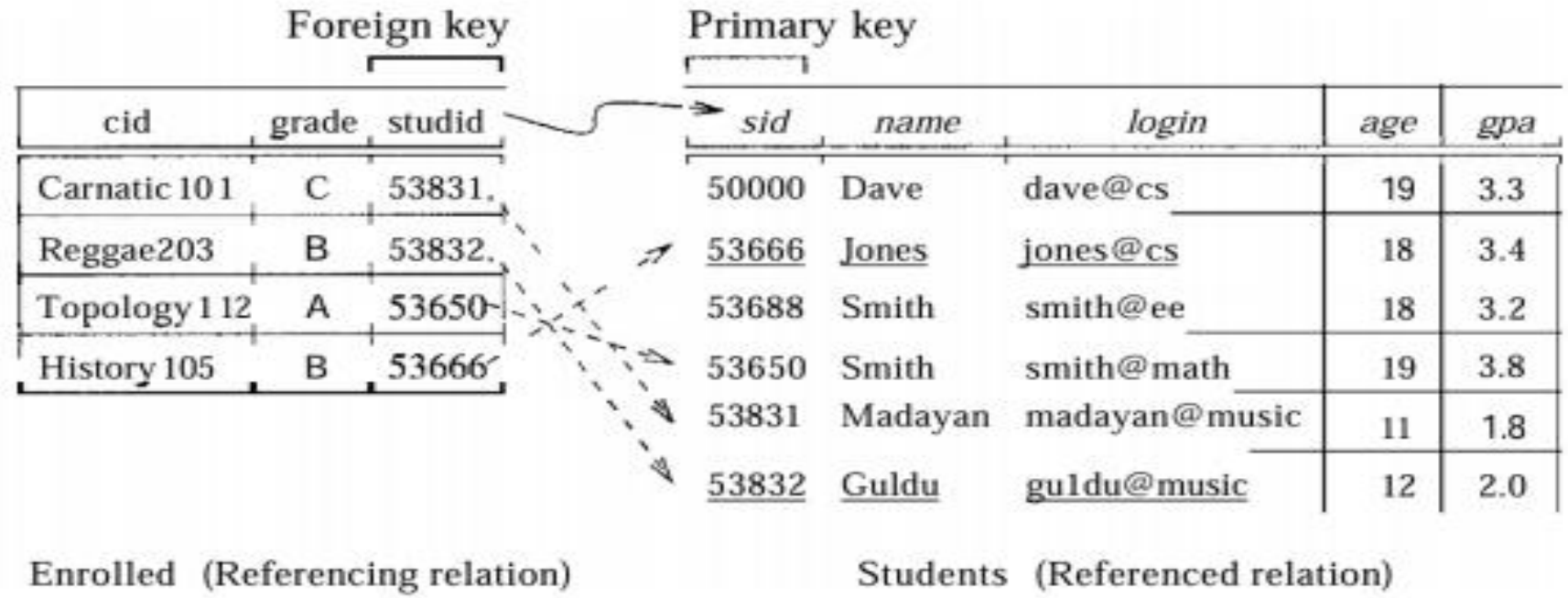
– properties on primary Keys

- Any relation in the database contains at least a primary key.
- It is not possible for a primary key to hold null value
- A primary key is a dataless, that is, its attribute values are never, or v ery rarely, changed

Key Type	Description	example
Super Key	Any set of attributes that uniquely identifies a row Any set of fields containing a (candidate) key	{StudentID, Email}
Candidate Key	Minimal super key (no extra attributes)	{Email}
Primary Key	One chosen candidate key for main identification	{StudentID}

Kinds of Integrity Constraints

- Referential Integrity(**Foreign Key)** Constraints – rule that states that a ny foreign key value (on the relation of the many side) MUST match a p rimary key value in the relation of the one side. (Or the foreign key can be null)



Kinds of Integrity Constraints

- For example: Delete Rules
  - Restrict – don't allow delete of "parent" side if related rows exist in "dependent" side
  - Cascade – automatically delete "dependent" side rows that correspond with the "parent" side row to be delet ed
  - Set-to-Null – set the foreign key in the dependent side to null if deleting from the parent side → not allowed for weak entities
  - NO ACTION-which means that the action (DELETE or U PDATE) is to be rejected.

Data manipulation

Structured Query Language(SQL)

You should be able to :

Data Definition, Constraints, and Schema Changes

- CREATE, DROP, and ALTER the descriptions of the tables (relations) of a database
- Referential Integrity Options.

Specifying Updates in SQL

- **INSERT**, **DELETE**, and **UPDATE**

Retrieval Queries in SQL

- **SELECT** <attribute list>
  FROM <table list>
  [**WHERE** <condition>]
  [**GROUP BY** <grouping attribute(s)>]
  [**HAVING** <group condition>]
  [**ORDER BY** <attribute list>]

Structured Query Language(SQL)

- Structured Query Language, SQL (pronounced “sequel,” or simply stated as S-Q-L).
- SQL is a language that can be used to work with a relational database.
- SQL is used to access, update, and manipulate the data stored in a database.
- **MySQL** was one of the first open-source database available in the market
- MySQL is a database that stores the existing data in a database in an organized manner.
- MySQL is a relational database that uses SQL to query a database .

Structured Query Language

SQL commands can be classified into four types

Data definition language (DDL)

Commands used to define a database, including

- CREATE – is used to create the database or its objects (like table, index, function,
- DROP – is used to delete objects from the database.
- ALTER- is used to alter the structure of the database.
- TRUNCATE–is used to remove all records from a table, including all spaces allocated for the records are removed.
- COMMENT –is used to add comments to the data dictionary.
- RENAME –is used to rename an object existing in the database.

DQL (Data Query Language) :

DML statements are used for performing queries on the data within schema objects. The purpose of DQL Command is to get some schema relation based on the query passed to it.

•SELECT – is used to retrieve data from the database.

Structured Query Language

Data manipulation language (DML)

Commands used to maintain and query a database,

- **INSERT** – is used to insert data into a table.
- **UPDATE** – is used to update existing data within a table.
- **DELETE** – is used to delete records from a database table.

Data control language (DCL)

DCL includes commands such as GRANT and REVOKE which mainly deals with the rights, permissions and other controls of the database system (access privileges to database.).

CREATE TABLE

Specifies a new base relation by giving it:

- a name &
- specifying each of its attributes &
- their data types (*INTEGER, FLOAT, DECIMAL(i,j), CHAR(n), VARCHAR(n)*)

Syntax: **`CREATE TABLE name_relation ( col1  data_type, col2  data_type,…)`**

DEPARTMENT

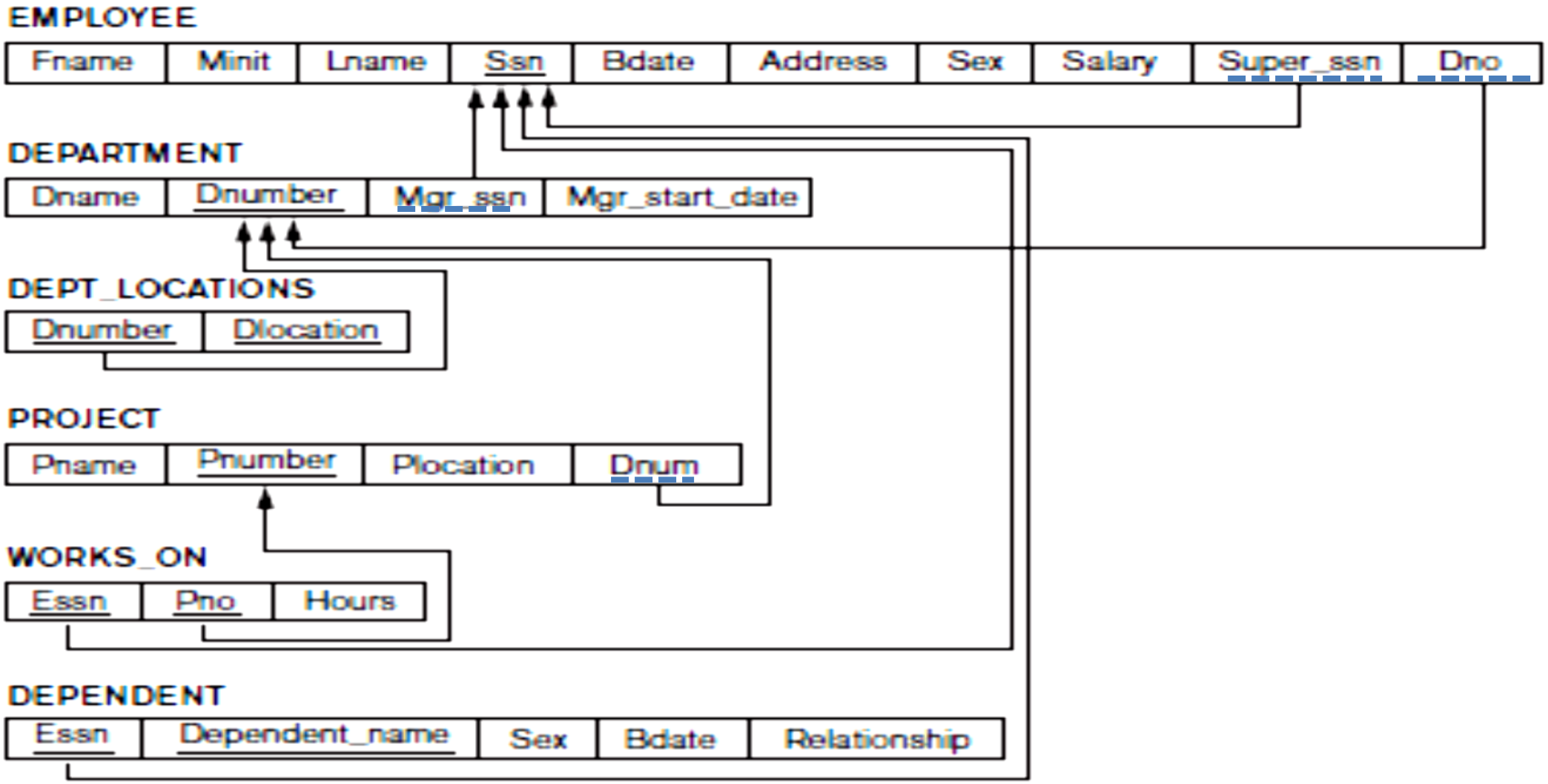
Dname	Dnumber

```
CREATE TABLE DEPARTMENT (
    DNAME          VARCHAR(10),
    DNUMBER        INTEGER);
```

Naming tables

- Start with Alphabetical
- Special characters as (_, #,) and numbers are allowed
- Not case sensitive (emp, Emp, EMP)
- Unique name within account
- Not a sql reserved words as Select, Update, ….
- Not more than 30 characters

Example: relational model for company



Specifying Attribute Constraints and Attribute Defaults

- A constraint NOT NULL may be specified on an attribute

```
CREATE TABLE DEPARTMENT (
    DNAME          VARCHAR(10)   NOT NULL,
    DNUMBER        INTEGER       NOT NULL,
```

- define a default value for an attribute

```
CREATE TABLE DEPARTMENT (
    DNAME          VARCHAR(10) DEFAULT 'Dep'
    DNUMBER        INTEGER       NOT NULL,
);
```

Specifying Attribute Constraints and Attribute Defaults

restrict attribute or domain values using the **CHECK** clause following an attribute or domain definition.

```
CREATE TABLE DEPARTMENT (
    DNAME          VARCHAR(10)   NOT NULL,
    Dnumber        INT           NOT NULL

    CHECK (Dnumber > 0 AND Dnumber < 21);
```

CREATE DOMAIN Dnumber AS INTEGER
CHECK (**Dnumber** > 0 **AND Dnumber** < 21);

CREATE TABLE : (primary and foreign key)

use the CREATE TABLE command for specifying:

- Primary Key attributes, **PRIMARY KEY(name of col.)**
- **Composite primary key** PRIMARY KEY (COL1, COL3, COL4)
- Secondary Keys, &
- Referential Integrity Constraints (Foreign Keys). **FOREIGN KEY (name of col.) REFERENCES (name of relation of PK)**

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

```
CREATE TABLE DEPT (
   DNAME       VARCHAR(10)  NOT NULL,
   DNUMBER     INTEGER      NOT NULL,
   MGRSSN      CHAR(9),
   MGRSTARTDATE    CHAR(9),
   PRIMARY KEY (DNUMBER),
   UNIQUE (DNAME),
   FOREIGN KEY (MGRSSN) REFERENCES EMP);
```

DROP TABLE

Used to remove a relation (base table) and its definition

The relation can no longer be used in queries, updates, or any other commands since its description no longer exists

Example:

```
DROP TABLE  DEPENDENT;
```

ALTER TABLE

The basic syntax of an ALTER TABLE command to:

- add a **New Column**

  ALTER TABLE table_name ADD column_name datatype;

- DROP COLUMN
  - ALTER TABLE table_name DROP COLUMN column_name;
- change the DATA TYPE
  - ALTER TABLE table_name MODIFY COLUMN column_name datatype;
- add a NOT NULL constraint
  - ALTER TABLE table_name MODIFY column_name datatype NOT NULL;
- ADD UNIQUE CONSTRAINT
  - ALTER TABLE table_name
  - ADD CONSTRAINT MyUniqueConstraint UNIQUE(column1, column2...);

ALTER TABLE: example

add an attribute called job to the Employee relation using ALTER command

```
ALTER TABLE EMPLOYEE ADD JOB VARCHAR(12);
```

– The new attribute will have NULLs in all the tuples of the relation right after the command is executed; hence, the NOT NULL constraint is not allowed for such an attribute

The database users must still enter a value for the new attribute JOB for each EMPLOYEE tuple.

– This can be done using the UPDATE command.

REFERENTIAL INTEGRITY OPTIONS

We can specify **RESTRICT, CASCADE, SET NULL or SET DEFAULT** on referential integrity constraints (foreign keys)

ON UPDATE CASCADE means that if the parent record is deleted, any child records are also deleted.

ON DELETE CASCADE means that if the parent record is deleted, any child records are also deleted.

RESTRICT means that any attempt to delete and/or update the parent will fail throwing an error.

SET NULL - Delete or update the row from the parent table, and set the foreign key column or columns in the child table to NULL

SET DEFAULT- specify a value to which to set the foreign key column(s) on an UPDATE or a DELETE

REFERENTIAL INTEGRITY OPTIONS

We can specify **RESTRICT, CASCADE, SET NULL or SET DEFAULT** on referential integrity constraints (foreign keys)

```
CREATE TABLE DEPT (
    DNAME          VARCHAR(10)   NOT NULL,
    DNUMBER        INTEGER       NOT NULL,
    MGRSSN         CHAR(9),
    MGRSTARTDATE   CHAR(9),
    PRIMARY KEY (DNUMBER),
    UNIQUE (DNAME),
    FOREIGN KEY (MGRSSN) REFERENCES EMP

    ON DELETE CASCADE);
```

REFERENTIAL INTEGRITY OPTIONS (continued)

```
CREATE TABLE EMP(
ENAME          VARCHAR(30) NOT NULL,
ESSN           CHAR(9),
BDATE          DATE,
DNO            INTEGER  DEFAULT 1,
SUPERSSN       CHAR(9),
PRIMARY KEY (ESSN),
FOREIGN KEY (DNO) REFERENCES DEPT
   ON DELETE SET DEFAULT
   ON UPDATE CASCADE,
FOREIGN KEY (SUPERSSN) REFERENCES EMP
   ON DELETE SET NULL
   ON UPDATE CASCADE);
```

Specifying Updates in SQL

There are three SQL commands to modify the database:

- **INSERT**
- **DELETE**
- **UPDATE**

INSERT

In its simplest form, it is used to add one or more tuples to a relation

Attribute values should be listed in the *same order* as the attributes were specified in the **CREATE TABLE** command

INSERT (contd.)

Example:

```
U1: INSERT INTO EMP
    VALUES      ('Richard','K','Marini', '653298653', '30-
                  DEC-52','98 Oak Forest,Katy,TX', 'M',
                  37000,'987654321', 4 )
```

An alternate form of INSERT specifies explicitly the attribute names that correspond to the values in the new tuple

- Attributes with NULL values can be left out

Example: Insert a tuple for a new EMPLOYEE for whom we only know the FNAME, LNAME, and SSN attributes.

```
U1A: INSERT INTO EMP (FNAME, LNAME, SSN)
     VALUES          ('Richard', 'Marini', '653298653')
```

INSERT (cont.)

Important Note:

- Only the constraints specified in the DDL commands are automatically enforced by the DBMS when updates are applied to the database

Another variation of INSERT allows insertion of *multiple tuples* resulting from a query into a relation

DELETE

Removes tuples(rows) from a relation

- Includes a WHERE-clause to select the tuples to be deleted
- Referential integrity should be enforced
- Tuples are deleted from only *one table* at a time (unless CASCADE is specified on a referential integrity constraint)
- A missing WHERE-clause specifies that *all tuples* in the relation are to be deleted; the table then becomes an empty table
- The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause

DELETE (contd.)

EMPLOYEE

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO

U4A: **DELETE FROM** EMPLOYEE
WHERE LNAME='Brown'

U4B: **DELETE FROM** EMPLOYEE
WHERE SSN='123456789'

U4C:
```
DELETE FROM  EMPLOYEE
WHERE     DNO IN( SELECT   DNUMBER
                  FROM          DEPARTMENT
                  WHERE   DNAME='Research')
```

U4D: **DELETE FROM** EMPLOYEE

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
	1	Houston
	4	Stafford
	5	Bellaire
	5	Sugarland
	5	Houston

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

UPDATE

Used to modify attribute values of one or more selected tuples

A WHERE-clause selects the tuples to be modified

An additional SET-clause specifies the attributes to be modified and their new values

Each command modifies tuples *in the same relation*

Referential integrity should be enforced

UPDATE (cont.)

Example: Change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively.

PROJECT

PNAME	PNUMBER	PLOCATION	DNUM

U5: **UPDATE** PROJECT
SET PLOCATION = 'Bellaire', DNUM = 5
WHERE PNUMBER=10

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

The following statements are correct??

INSERT
INTO **employee** (fname,lname,SSN,Bdate)
VALUES ('Jann', 'jams',123456789, 1990-01-09)

INSERT
INTO **employee** (fname,lname,SSN,Bdate)
VALUES ('Jann', 'jams',NULL, 1990-01-09)

UPDATE **employee S**
SET **S**.*SSN* = 123456789
WHERE **S**.*SSN* = 333445555

More examples

1. creates a table called "Persons" that contains five columns: PersonID, LastName, FirstName, Address, and City all of them have datatypes string with variable length with max size 255

```
CREATE TABLE Persons (
    PersonID varchar(255),
    LastName varchar(255),
    FirstName varchar(255),
    Address varchar(255),
    City varchar(255)
);
```

- **CREATE TABLE IF NOT EXISTS**

```
CREATE TABLE IF NOT EXISTS Persons
(
    PersonID varchar(255),
    LastName varchar(255),
    FirstName varchar(255),
    Address varchar(255),
    City varchar(255)
);
```

More examples(cont.)

2. Write a SQL statement to create a table named jobs including columns job_id(Not Null), job_title(Not Null), min_salary, max_salary and check whether the max_salary amount exceeding the upper limit 25000.

```
CREATE TABLE IF NOT EXISTS jobs (
JOB_ID varchar(10) NOT NULL ,
JOB_TITLE varchar(35) NOT NULL,
MIN_SALARY decimal(6,0),
MAX_SALARY decimal(6,0)
CHECK(MAX_SALARY<=25000)
);
```

More examples (cont.)

2. Write a SQL statement to rename the table countries to country_new

- ALTER TABLE countries RENAME country_new;

3. Write a SQL statement to add a columns ID of the table locations.

- ALTER TABLE locations ADD ID INT ;

4. Write a SQL statement to add a columns ID as the first column of the table locations.

- ALTER TABLE locations ADD ID INT FIRST;

5. Write a SQL statement to add a column region_id after state_province to the table locations.

- ALTER TABLE locations ADD region_id INT AFTER state_province;

More examples (cont.)

5. Write a SQL statement to change the name of the column state_province to state , keeping the data type and size same.

```
ALTER TABLE locations Rename state_province  to state ;
```

6. Write a SQL statement to add a foreign key constraint named fk_job_id on job_id column of job_history table referencing to the primary key job_id of jobs table

```
ALTER TABLE job_history ADD CONSTRAINT fk_job_id
FOREIGN KEY (job_id) REFERENCES jobs(job_id)
 ON UPDATE RESTRICT
ON DELETE CASCADE;
```

```
ALTER TABLE job_history DROP CONSTRAINT fk_job_id
```

More examples (cont.)

6. Write a SQL statement to insert a record with your own value into the table countries with attributes (COUNTRY_ID, COUNTRY_NAME, REGION_ID) against each columns . (Hint: the values of type string except REGION_ID) of type integer)

```
INSERT INTO countries VALUES ('C1',‘Egypt',1001);
```

7. Write a SQL statement to insert one row into the table countries against the column country_id and country_name with values C5 and USA. (Hint: the values of type string)

```
INSERT INTO countries (country_id,country_name) VALUES('C5',‘USA');
```

8. Write a SQL statement to insert 3 rows by a single insert statement with your own value into the table countries with attributes (COUNTRY_ID, COUNTRY_NAME, REGION_ID) against each columns . (Hint: the values of type string except REGION_ID) of type integer)

```
INSERT INTO countries VALUES('C0001','India',1001), ('C0002','USA',1007),('C0003','UK',1003);
```

More examples (cont.)

9. Write a SQL statement to change the email and commission_pct column of employees table with 'not available' and 0.10 for those employees whose department_id is 110.

```
UPDATE employees
 SET email='not available', commission_pct=0.10
WHERE department_id=110;
```

10. Write a SQL statement to change salary of employee to 8000 whose ID is 105, if the existing salary is less than 5000.

```
UPDATE employees SET SALARY = 8000 WHERE employee_id = 105 AND salary < 5000;
```

Transactions & Constraints

A constraint in deferred mode

```
CREATE TABLE Students (
sid CHAR(20),
name CHAR(30),
login CHAR(20),
age INT,
honors CHAR(10) NOT NULL,
gpa REAL,
PRIMARY KEY (sid),
FOREIGN KEY (honors) REFERENCES Courses (cid))
```

```
CREATE TABLE Courses (
cid CHAR(10),
cname CHAR(10),
credits INT,
grader CHAR(20) NOT NULL,
PRIMARY KEY (cid),
FOREIGN KEY (grader) REFERENCES
Students (sid))
```

SQL allows a constraint to be in DEFERRED or IMMEDIATE mode as:

SET CONSTRAINT ConstntintFoo DEFERRED


THANK
YOU


Any
Question?